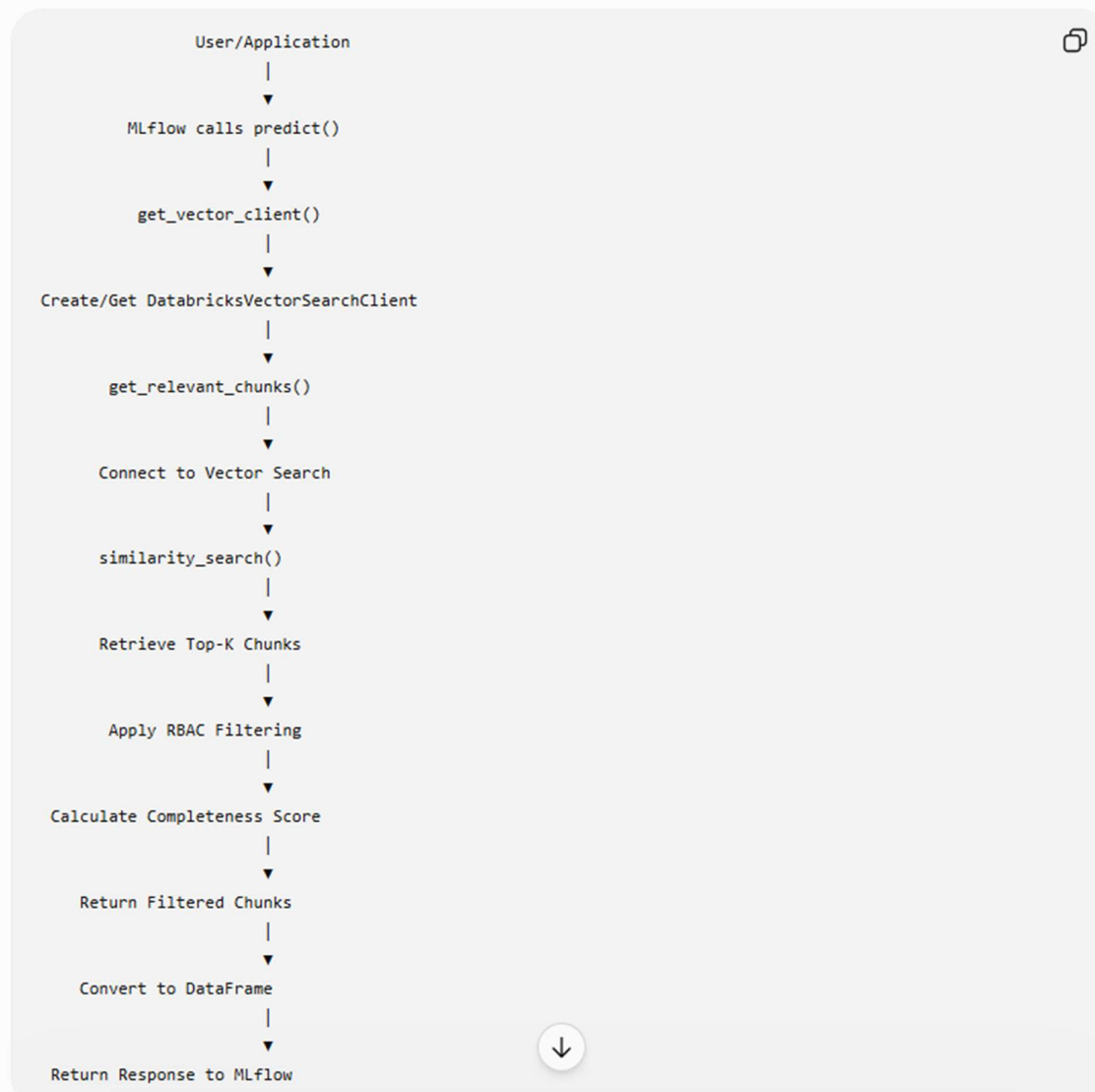


High-Level Flow



Step-by-Step Execution (According to Your Code)

Step 1: MLflow Creates the Model

When this file is loaded, these lines execute:

```
retriever_model = SimpleRetriever()
```

```
mlflow.models.set_model(retriever_model)
```

What happens?

SimpleRetriever Object Created



MLflow now knows

"This is the model to serve."

Nothing is searched yet.

Step 2: User Sends a Query

Suppose the user asks:

How many casual leaves are allowed?

The request reaches the MLflow endpoint.

Example input:

model_input =

query : Leave Policy

top_k : 5

username : Rahul

country : India

Step 3: MLflow Automatically Calls predict()

MLflow internally executes

`predict(context, model_input)`

Execution enters here.

```
class SimpleRetriever(mlflow.pyfunc.PythonModel):
```

```
    def predict(...):
```

Think of this as

API Request



predict()

Step 4: Get the Vector Client

Inside predict()

```
vector_client = get_vector_client()
```

Execution jumps here

```
def get_vector_client():
```

What does it do?

Checks

Has a client already been created?

|

Yes / No

If No

Creates

```
DatabricksVectorSearchClient(...)
```

This runs

```
__init__()
```

Inside constructor

Store

endpoint

vector index

workspace url

identity

Object created

client

|

|— endpoint

|— vector index

|— workspace

└— identity

If already created

Reuse the existing client.

This avoids reconnecting every request.

Step 5: Back to predict()

Now

vector_client

is ready.

Loop starts

for idx,row in model_input.iterrows():

Suppose

Only one row

Extract

query

top_k

username

country

Step 6: Call Retriever

Now

```
result = vector_client.get_relevant_chunks(...)
```

Execution jumps here

```
get_relevant_chunks()
```

This is where the real retriever starts.

Step 7: Create Databricks Connection

Inside

```
vsc = VectorSearchClient(...)
```

Connection created.

Think

Python

↓

Databricks Vector Search

Step 8: Open Vector Index

```
index_client = vsc.get_index(...)
```

Now

Python knows

Which Vector Database

to search.

Step 9: Search the Index

Most important line

```
results = index_client.similarity_search(...)
```

Execution

User Query

↓

Embedding

↓

Compare with all vectors

↓

Top K Similar Chunks

Suppose

Chunk A

Chunk B

Chunk C

Chunk D

Chunk E

returned.

Step 10: Convert Results

Databricks returns

Rows

Columns

The code converts them into

```
[  
  {  
    "title":...,  
  
    "content":...,  
  
    "source":...  
  },  
  
  ...  
]
```

This becomes

chunks

Step 11: Apply RBAC

Now

```
rbac_chunks = self.rbac_function(...)
```

Execution jumps here

```
rbac_function()
```

Current implementation

```
return chunks
```

So

Input

↓

Chunks

↓

Output

↓

Same Chunks

No filtering.

In future

Chunks

↓

Permission Check



Remove Unauthorized Chunks



Allowed Chunks

Step 12: Calculate Completeness

Now

`score = self.completeness(...)`

Execution jumps here

`completeness()`

Suppose

Retrieved

10

Allowed

8

Returns

8/10

=

0.8

Step 13: Return from Retriever

Now

```
return {
```

```
    data,
```

```
    completeness_score,
```

```
    status_code
```

```
}
```

Execution returns

back to

```
predict()
```

Step 14: Build Response

Inside

```
responses.append(result)
```

If multiple rows

Repeat.

Step 15: Convert to DataFrame

Finally

```
return pd.DataFrame(responses)
```

MLflow always wants

DataFrame

instead of Python dictionary.

Step 16: MLflow Returns Response

MLflow sends

```
{  
  "data":[...],  
  
  "status_code":200,  
  
  "completeness_score":0.8  
}
```

back to

Application



Prompt Builder



Azure OpenAI



Answer

Complete Code Flow

File Starts

|



Create SimpleRetriever

|



Register with MLflow

|

User Sends Query

|



predict()

|



get_vector_client()

|



Create DatabricksVectorSearchClient

(if not already created)

|



__init__()

|



Store endpoint/index/workspace

|



Return client

|



predict()

|



Extract query/top_k/user

|



get_relevant_chunks()

|



Connect Vector Search

|



Open Index

|



similarity_search()

|



Retrieve Top-K Chunks

|



Convert Results

|



rbac_function()

|



Filter Unauthorized Chunks

(Currently returns all)

|



completeness()

|



Calculate Score

|



Return Result

|



predict()

|



Build DataFrame

|



Return to MLflow

|



Application Receives Chunks

|



LLM Uses Those Chunks

|



Final Answer

One important observation

Notice the flow carefully:

- **predict() is the entry point** because MLflow calls it.
- **get_vector_client()** creates or reuses the retriever object.
- **get_relevant_chunks()** contains the core retrieval logic.
- **rbac_function()** filters the retrieved chunks based on access permissions (currently it's a placeholder and returns all chunks).
- **completeness()** calculates how much of the retrieved information remains after filtering.
- The retriever returns the chunks to the caller. **It does not call the LLM.**

So the complete RAG flow is:

Documents

|



Chunking

|



Embeddings

|



Databricks Vector Search

|

User Question

|



Your Retriever Code

|



Relevant Chunks

|



Prompt Builder (another component)

|



Azure OpenAI / GPT

|



Final Answer

This distinction is important: **your code implements only the Retriever layer**, while **the Prompt Builder and LLM Generator are separate components that use the retriever's output.**

MLflow is the model am i right what will be the work

This is a very common confusion. The answer is:

No. MLflow is not the model itself.

MLflow is a platform/framework that manages and serves models.

Think of it like this.

Real-Life Example

Imagine you have a restaurant.

Chef

|



Cooks Food

Who is the chef?

Your Retriever Code

Who is the restaurant?

MLflow

The restaurant does **not** cook food.

The restaurant:

- Accepts customer orders
- Sends the order to the chef
- Receives the cooked food
- Gives it back to the customer

MLflow works in exactly the same way.

In your code

Your actual model (retriever) is

```
class SimpleRetriever(mlflow.pyfunc.PythonModel):
```

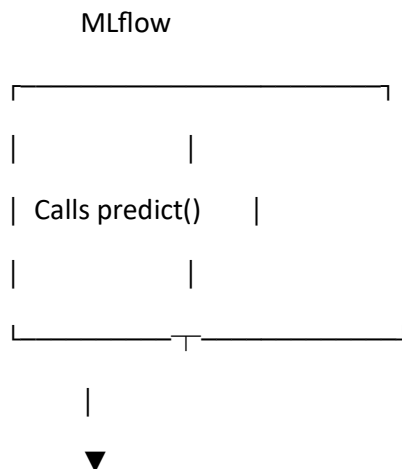
Inside this class, the important method is

```
def predict(...)
```

This SimpleRetriever is **your model**.

MLflow is only the framework that hosts it.

Architecture



SimpleRetriever

|



DatabricksVectorSearchClient

|



Databricks Vector Search

What is the work of MLflow?

MLflow has several responsibilities.

1. Model Packaging

Suppose you write

```
class SimpleRetriever(mlflow.pyfunc.PythonModel):
```

MLflow packages it as a deployable model.

Without MLflow, it's just a normal Python class.

With MLflow, it becomes something that can be served.

2. Model Serving

Suppose someone sends

POST /invocations

MLflow receives it.

Instead of you writing API code,

MLflow automatically calls

```
predict()
```

Flow

User



REST API



MLflow



predict()



Return Response

3. Model Registry

MLflow stores versions.

Example

Retriever v1

Retriever v2

Retriever v3

You can always go back.

4. Deployment

MLflow makes deployment easier.

Instead of writing

Flask

FastAPI

Gunicorn

Docker

MLflow can deploy your model.

5. Standard Interface

MLflow expects every model to have

`predict()`

That's why your class inherits from

`mlflow.pyfunc.PythonModel`

What happens when a request comes?

Suppose your application sends

```
{  
  "query": "Leave Policy",  
  "top_k": 5  
}
```

Flow

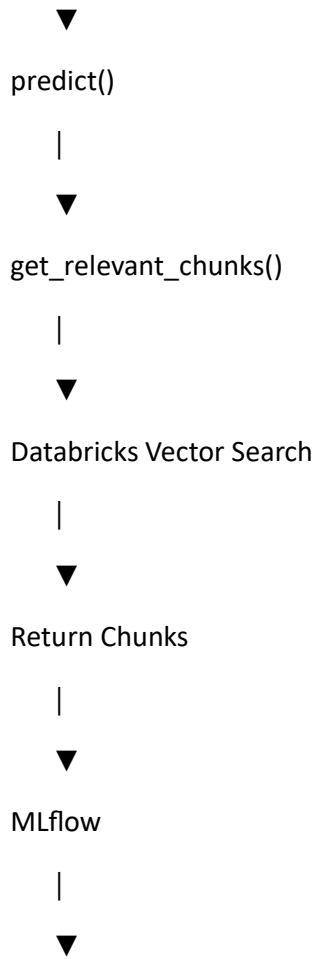
Application

|



MLflow Server

|



Application

Notice something?

MLflow never searches the vector database.

Your retriever does.

In your code

These two lines

```
retriever_model = SimpleRetriever()
```

```
mlflow.models.set_model(retriever_model)
```

mean

Create Retriever Object



Tell MLflow

"This is the model you should serve."

Is MLflow doing the retrieval?

No.

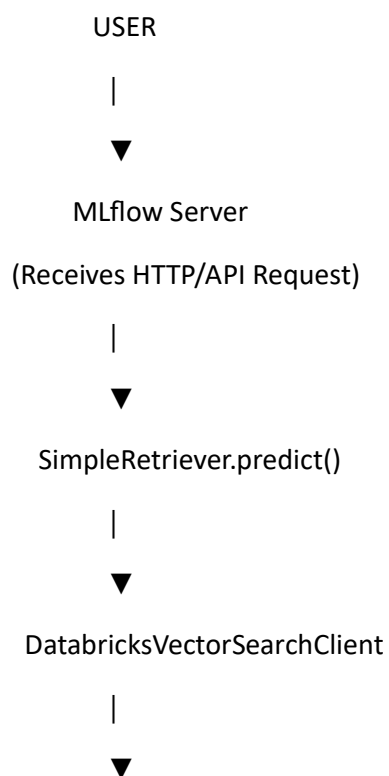
The retrieval is done here:

```
result = vector_client.get_relevant_chunks(...)
```

MLflow doesn't know how to retrieve documents.

It simply calls your code.

Complete Picture



similarity_search()

|



Relevant Chunks Found

|



Return to MLflow

|



Return to User

Simple analogy

Real World Your Project

Restaurant MLflow

Chef SimpleRetriever

Cooking get_relevant_chunks()

Kitchen Databricks Vector Search

Customer User/Application

The restaurant (**MLflow**) doesn't cook. It manages the process.

The chef (**your retriever**) does the actual work.

One small correction to your understanding

You asked:

"MLflow is the model, am I right?"

A more accurate statement is:

-  **MLflow is the model.**
-  **MLflow is the framework that packages, manages, and serves your model.**

-  **SimpleRetriever is your model.**
-  **DatabricksVectorSearchClient contains the core retrieval logic that the model uses.**

This distinction is important because in Databricks you'll often see many different MLflow models (retrievers, classifiers, forecasting models, etc.) all being managed by the same MLflow platform.